DRIVIX SMART PARKING

Full-Stack Technology Stack & Architecture Specification Report

1. EXECUTIVE SUMMARY

Drivix is a premium, high-concurrency smart parking and driver utility platform. The system connects drivers with real-time parking space inventories, dynamic slot allocation algorithms, automated ANPR (Automatic Number Plate Recognition) gate access, and value-added utilities (FASTag recharge, e-Challan status, and utility bill payments). Built using a modern, decoupled layered architecture, the stack is engineered for speed, typesafety, and memory caching scalability.

2. FRONTEND TECHNOLOGY STACK

The Drivix frontend is compiled as a high-performance Single Page Application (SPA) leveraging Vite and React. It integrates premium styling guidelines (glassmorphic dark theme, custom responsive grid layouts) and real-time state listeners.

Technology / Package	Purpose & Business Case
React 19 & React DOM	Declarative component-based UI framework for state reactivity.
Vite 8 & React Plugin	Modern build engine providing hot module replacement (HMR) and fast assets bundling.
TypeScript 7	Provides strict compile-time typesafety across data structures and component interfaces.
Zustand 5	Lightweight, high-performance global state manager tracking slot details and bookings.
Framer Motion 12	Premium hardware-accelerated animations, transitions, and loading sequences.
Socket.io Client	Establish bi-directional, real-time connection with backend servers for live slot counts.
Firebase Client SDK	Handles client-side Firebase Auth, phone/Google OAuth integration, and gate simulator.
React Router DOM 7	Declarative router managing routes (home, slots layout, booking form, admin views).
@react-google-maps/api	Google Maps SDK integration for listing and selecting parking hubs geographically.
Three.js & React Icons	Interactive 3D graphic animations and custom vector design iconography.
html-to-image & Qrcode	Generates and converts booking receipt components into downloadable ticket images with QR codes.

3. BACKEND TECHNOLOGY STACK

The Drivix backend is built using a decoupled layered architecture (Controller ➡■ Service ➡■ Repository) on Node.js/Express, utilizing Redis caching and Socket.io for live updates.

Technology / Package	Purpose & Business Case
Node.js & Express	Minimal, fast web framework routing and executing API REST endpoints.
Mongoose & MongoDB	NoSQL database schema modeller for User, Booking, Slot, and Location logs.
Redis Client v4	High-speed in-memory data store caching floor capacity metrics (60s TTL).
Socket.io Server	Broadcasting floor capacity updates globally to clients on database booking changes.
JsonWebToken (JWT)	Stateless user authentication authorization via encrypted Bearer header tokens.
Bcrypt.js Hashing	One-way cryptographic salting and hashing security for user password records.
Nodemailer SMTP	Configured SMTP mail provider delivering registration codes to user Gmail accounts.
Jest & Supertest	Automated integration test suites running native ESM code tests prior to builds.
Helmet & Cors	Security headers protection and cross-origin resource sharing policy managers.
Dotenv & Morgan	System environment file loaders and HTTP logging middleware logger.

4. SYSTEM ARCHITECTURE SPECIFICATIONS

Drivix is built around three core architectural tenets:

- **Layered Decoupled Architecture:** Separates route controllers from business services and Mongoose queries (Repositories), allowing isolation of DB lookups and direct unit/integration testing.
- **Redis Cache-Aside Model:** High-volume GET requests are intercepted by Redis. Database write operations automatically call in-memory invalidation hooks, preventing stale cache data.
- **Websocket Event Broadcast:** Eliminates HTTP polling. When a slot state updates, Socket.io broadcasts the changes instantly to all active client floor plans.